

Phase 1: Model

1.1 Mission & Goal

Build the mathematical **brain** of the printer. You are creating a **Logic Engine** — an engine that models how components age, fail, and interact under environmental and operational stress.

This engine ingests conditions as input and produces the state of every component for the digital replica. You are **not** building a UI or a simulation loop here. You are building a **Mathematical Engine** that will serve as the “source of truth” for the Digital Twin in all subsequent phases.

1.2 Printer Subsystems & Component Map

The HP Metal Jet S100 is decomposed into **three core subsystems**. You must model at least one component from each subsystem — that gives you three degradation models, one per subsystem. The more components you add, the richer and more realistic your twin will be.

Subsystem	Component	Requirement	Primary Function
Recoating System	Recoater Blade	Model at least one	Spreads a thin, even layer of metal powder.
	Recoater Drive Motor	Optional	Drives the motion of the blade assembly.
	Linear Guide / Rail	Optional	Ensures smooth, straight-line movement.
Printhead Array	Nozzle Plate	Model at least one	Precisely ejects binder liquid onto the powder.
	Thermal Firing Resistors	Optional	Controls the temperature of the printhead.
	Cleaning Interface	Optional	Wipes the nozzle to prevent clogging.
Thermal Control	Heating Elements	Model at least one	Maintains the optimal build temperature.
	Temperature Sensors	Optional	Provides feedback on thermal stability.
	Insulation Panels	Optional	Reduces heat loss to the environment.

You can add as many additional components as you wish — the more you model, the more accurately your twin reflects real-world behaviour. If you are unsure which optional components to add, ask a Volunteer for guidance.

1.3 Component Modeling Guides (Physics Hints)

Since you are modeling a digital version of a physical machine, use these “physics hints” to guide your logic.

How HP Metal Jet Printing Works

The **HP Metal Jet S100** uses **Binder Jetting** technology to 3D print metal parts. It is quite different from filament-based or laser sintering printers.

The Metal Binder Jetting Process

Step-by-Step

1. Powder Bed Preparation

A thin layer of metal powder (e.g., stainless steel or other alloys) is spread evenly across the build platform by the re-coating carriage.

2. Binder Jetting

A printhead (similar to an inkjet) selectively jets a liquid **binding agent** onto the powder, following the cross-section of each layer. The binder glues the metal particles together.

3. Layer Repetition

The platform lowers slightly, a new powder layer is spread, and the process repeats — layer by layer — until the full 3D “green part” is built.

4. Curing

The build box is heated to cure and strengthen the binder, consolidating the green part before extraction.

5. Sintering

The green part is placed in a furnace. The binder burns off and the metal particles fuse together under heat, producing a dense, solid metal component.

6. Extraction & Post-Processing

Loose powder is removed (and recycled). The finished metal part is extracted and can be further processed (e.g., machining, surface finishing).

Subsystems

Subsystem A: Re-coating System

- **The Re-coater Blade:** Suffers from **Abrasive Wear**. As it moves through powder, it loses thickness and smoothness.
 - *Hint:* Consider how “Contamination” (an input vector) might accelerate this wear.
- **The Drive Motor & Rails (Optional):** Subject to **Mechanical Fatigue**.
 - *Hint:* High “Production Volume” should increase the probability of motor stall or rail misalignment.

Subsystem B: Printhead Array

- **The Nozzle Plate:** Susceptible to **Clogging** and **Thermal Fatigue**.
 - *Hint:* If “Temperature Stress” is outside optimal bounds, the likelihood of a clog should increase.
- **Cleaning & Thermal Interface (Optional):**
 - *Hint:* The efficiency of the cleaning mechanism determines how long the Nozzle Plate remains in a FUNCTIONAL state.

Subsystem C: Thermal Control

- **Heating Elements:** Follow the principle of **Electrical Degradation**.
 - *Hint:* As they age, they may require more energy to maintain the same temperature.
- **Insulation & Sensors (Optional):**
 - *Hint:* If insulation degrades, the heating elements must work harder, creating a feedback loop of accelerated wear.

1.4 Implementation Guidance: Rule-Based vs. Data-Driven

Your engine can be implemented in two ways:

1. **Rule-Based (Deterministic):** You use classical mathematical formulas to calculate the state. This is the standard approach.
2. **Data-Driven (Probabilistic/AI):** Instead of a fixed formula, you use a trained model (e.g., a Neural Network or Regression model) to predict the state based on the input vectors.

Regardless of your choice, your engine MUST adhere to the Data Contract and must be deterministic (two runs with the same inputs must provide the same output).

1.5 The Data Contract (Inputs & Outputs)

To ensure your engine can be integrated into the Phase 2 simulation, you must adhere to the following data structure:

1.5.1 Expected Input Drivers

The drivers are the input values that change over time and have a direct impact on the state of the components. Your engine must accept a set of “Environmental & Operational Vectors”:

- **Temperature Stress:** (e.g., Ambient temperature in Celsius).
- **Humidity/Contamination:** (e.g., Air moisture or powder purity).
- **Operational Load:** (e.g., Total print hours or cycles completed).
- **Maintenance Level:** (e.g., A coefficient representing how well the machine is being cared for).

1.5.2 Expected Outputs (The Telemetry)

Your engine must return a structured “State Report” for every component modeled:

- **Health Index:** A normalized value (e.g., 0.0 to 1.0) representing the component’s remaining life.
- **Operational Status:** A categorical state (e.g., FUNCTIONAL, DEGRADED, CRITICAL, or FAILED).
- **Component Metrics:** Custom physical variables defined by your model (e.g., if modeling a blade, this could be ‘thickness’; if a resistor, ‘resistance’).

1.6 Minimum Requirements

To successfully complete this phase, your engine must meet these minimum requirements:

1. **Model at least one component per subsystem** (Recoating, Printhead Array, Thermal Control) and connect each to at least one relevant input driver.
2. **Apply** at least two standard mathematical failure models (e.g., Exponential Decay or Weibull Distribution) to represent component aging.
3. **Ensure** all components react dynamically to at least the four input drivers listed in section 1.5.1.

1.7 Go Further

The minimum gets you a working model. These ideas can make it exceptional. They are starting points — feel free to take them further or invent your own.

1. **Cascading Failures** What if a degrading recoater blade starts contaminating the powder, which then clogs the nozzle plate? Model components that react to each other’s degradation across subsystems. A failure in one part of the machine accelerates wear in another — just like in real life.
2. **Stochastic Realism** Real machines don’t degrade on a smooth curve. Add random environmental shocks, sensor drift, and probabilistic failure events to make your simulation unpredictable and alive. A spike in contamination, an unexpected temperature surge — how does your model cope?
3. **Maintenance as an Input** What happens when a technician intervenes? Model maintenance actions as inputs to the engine: partial health recovery, reset of degradation curves, cooldown effects after servicing. This sets up a fascinating question for Phase 2 — *when* is the right moment to trigger maintenance?

4. AI-Powered Degradation Model Swap out a hand-tuned formula for a machine learning model. Can a neural network learn the degradation curve from synthetic data better than a classical equation? Try replacing one component's rule-based model with a trained regressor or LSTM.

5. Live Environmental Data Instead of synthetic input values, plug in a real weather API. Let actual temperature and humidity data from a city of your choice drive the simulation. How does your printer hold up in Barcelona vs. Phoenix?

1.8 Deliverables & Evaluation

What to deliver

- **3 degradation models coded** — at least one per subsystem, each using at least one input driver.
- **Explanation of your modeling approach** — describe how each degradation model was designed and implemented. A short written or verbal walkthrough is sufficient.

How it will be evaluated

Criterion	Description
Rigor	Degradation and failure models are logically consistent and physically plausible.
Systemic Interaction	Components react realistically to environmental and operational input drivers.
Complexity & Innovation (<i>bonus</i>)	Advanced behaviours such as cascading failures, stochastic models, or AI-driven degradation.
Realism & Fidelity (<i>bonus</i>)	Model behaviour closely reflects how the real machine would degrade over time.

Tip: The more input drivers you connect to a component, the richer its behaviour and the higher it can score on Systemic Interaction. Make sure you can explain *why* each driver affects each component — that reasoning is part of the evaluation.

Align the implementation with the scoring criteria in `scoring/scoring.md`.

[Back to Main Brief](#)